

Report HW1: Rudy - a small web server

Bruna Fernandes Gabriele Giustizieri

September 9, 2026

1 Introduction

This assignment explores important distributed systems topics by implementing Rudy, a small web server in Erlang. Through it, we worked directly with the procedures for using a socket API, the structure of a server process and the HTTP protocol, which are the basis of most networked applications.

These concepts matter because client-server communication and performance under load are central challenges in distributed systems. A real server that can't handle concurrent requests is probably not very useful.

We implemented an HTTP parser, a working web server and a benchmark program, then measured performance under different conditions and used the results to guide further improvements, discussed later in this report.

2 Main problems and solutions

One of the main problems we identified during development was that our sequential server could not scale with the number of concurrent clients. As more clients sent requests simultaneously, response times grew substantially, since each request had to wait for every previous one to be fully handled before being served. To address this, we implemented a simple multi-threaded version of the server that spawns a new process for each incoming client connection, allowing requests to be handled concurrently rather than one at a time. The benchmarks in the evaluation section confirm that this change significantly improves performance under concurrent load. We recognize, however, that this is not necessarily the optimal solution, since spawning a new process per request introduces additional waiting time. A more scalable approach would be to maintain a pool of pre-spawned handler processes and assign incoming requests to them as they arrive, avoiding repeated process creation (though we did not implement this within the scope of this assignment).

The second problem we tackled was that an HTTP request is not guaranteed to arrive in a single block, as it may be split across multiple blocks,

which then need to be concatenated before the request can be parsed correctly. Our solution accumulates incoming data and scans it for a double CRLF, which marks the end of the header section. If this sequence is not yet present, the server keeps reading additional blocks until it is found. To validate this solution, we wrote a test that sends a request split into two parts with a delay in between. With the original parsing logic, this caused the server to crash, since it attempted to parse an incomplete request before the headers had fully arrived. With the updated logic, the server correctly waits for and assembles the full request before parsing, producing a valid response.

3 Evaluation

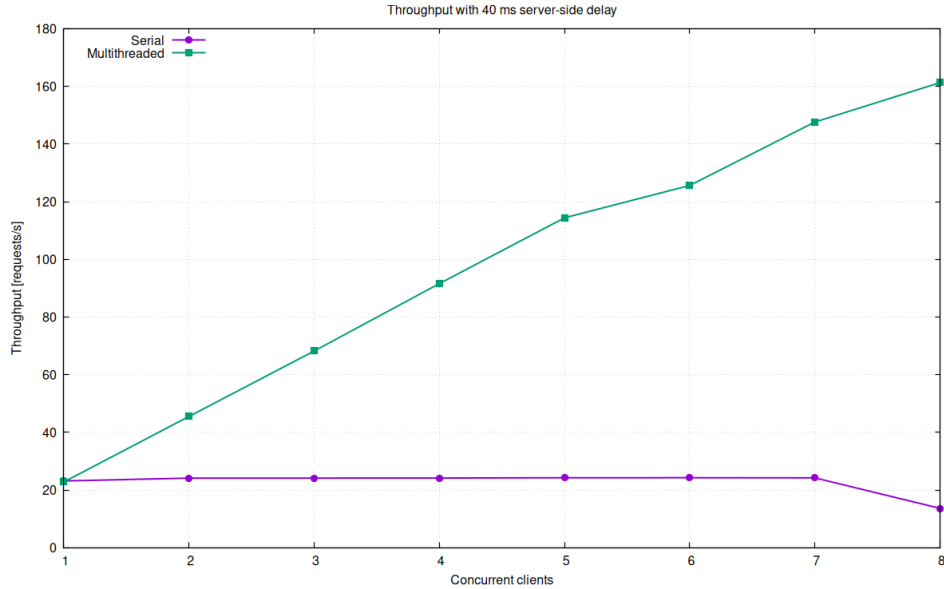


Figure 1: Throughput comparison between serial and multithreaded server with 40ms delay.

When we ran the benchmark program with the sequential server, we observed a throughput of approximately 24 requests per second. This is consistent with the artificial 40 ms delay introduced in the request handling, since it imposes a theoretical upper bound of $\frac{1}{0.040s} = 25\text{requests/s}$. The slightly lower value we measured is likely due to additional overhead from parsing, connection setup, etc.

This artificial delay proves to be highly significant for overall performance. Without it, requests could be parsed and answered almost instantly, and the server’s throughput would be limited mainly by network and pro-

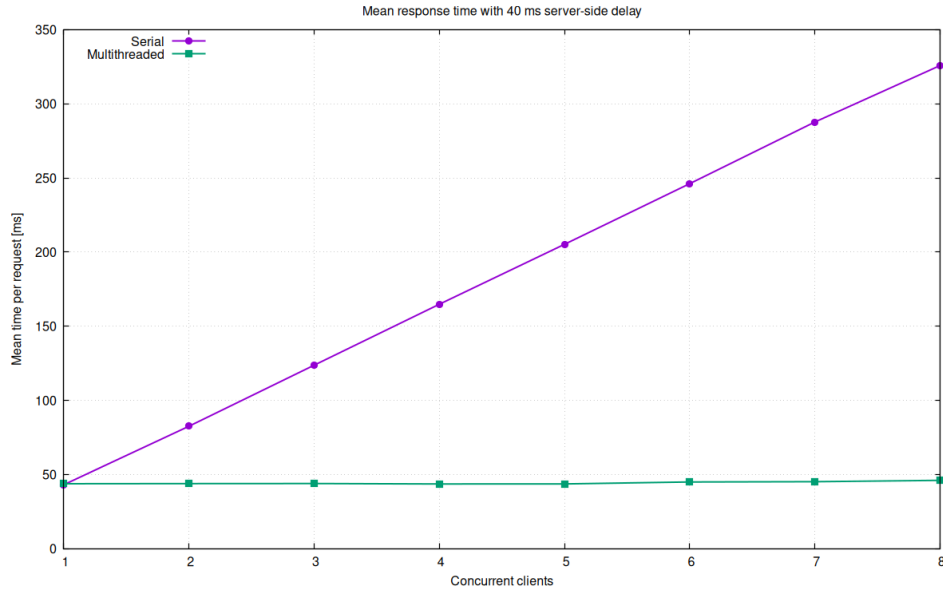


Figure 2: Response time comparison between the serial and multithreaded server with 40 ms delay.

cessing overhead rather than any fixed cost per request. With the delay in place, however, each client costs the server a little over 40 ms of dedicated time, and since our server handles requests sequentially, this cost accumulates linearly with the number of clients. As a result, the delay becomes the dominant bottleneck as concurrent load increases, which is precisely the problem that motivates the multi-threaded improvement described earlier.

4 Conclusions

In this assignment, we learned how a simple web server can be built in Erlang using sockets, HTTP parsing, and server processes. The experiments showed how strongly a sequential design limits performance under concurrent load, especially when request handling includes waiting time.

By introducing concurrency with Erlang processes, we significantly improved the server's ability to handle multiple clients at the same time. We also improved the robustness of the HTTP parsing by handling requests that arrive in multiple blocks. Overall, the assignment was useful for understanding how concurrency, communication, and implementation choices directly affect the performance and behavior of a distributed system.